ADVENTIST UNIVERSITY OF CENTRAL AFRICA

Big Data Analytics

Smart Energy Grid Monitoring System

Submitted by:

SHYAKA Hamza – 101230

KAGAME Fred – 101231

GAKUBA GATERA Felicien - 101885

1. INTRODUCTION

Smart Energy Grid Monitoring System requires strong data pipelines capable of handling massive volumes of high-frequency time-series data. This technical report details the implementation and performance evaluation of a specialized three-tier data framework designed to ingest, store, and visualize smart energy meter telemetry.

To simulate a realistic IoT deployment, 1,000 smart meters generate continuous electrical metrics, modeling real-world daily pattern human behavior curves characterized by distinct morning and evening usage peaks. Telemetry data is efficiently packaged and transmitted using **MQTT (Message Queuing Telemetry Transport)**, a lightweight, publish-subscribe network protocol ideal for resource-constrained remote sensors.

These data are processed by a central ingestion engine and committed to a local, native installation of **TimescaleDB**, an open-source time-series database built on PostgreSQL that leverages automated time-series partitioning (hypertables) to maintain heavy data traffic. Finally, real-time insight is achieved through a native **Grafana** server querying localized data pools over port 5432.

The primary objective of this report is to analyze how database performance scales under various data partitioning (chunking) strategies, evaluate the efficiency of database compression, and demonstrate the massive computational advantages of using continuous aggregations over raw dataset scans.

2. INFRASTRACTURE OVERVIEW & IMPLEMENTATION DETAILS

2.2 System Architecture

The system is built in three independent layers to isolate data generation, database management, and analytics presentation:

- **Data Simulation Layer**: A Python batch-insertion engine that simulates real-world grid demand by generating pseudo-random energy readings modeled after peak morning and evening consumer habits.
- **Storage & Processing Layer**: A local, native installation of TimescaleDB. This layer optimizes time-series ingestion and disk space through automated partitioning (hypertables) and native data compression.
- **Visualization Layer**: A native Grafana server running locally, which establishes a direct connection over port 5432 to query the localized TimescaleDB data pools in real time.

2.3 Schema Design

To ensure strict data integrity and efficient storage layout, the hypertable enforces the following explicit data types:

- **meter_id:** Defined as VARCHAR(10) to uniquely identify each individual hardware unit and ensure the digits do not exceed 10.
- **time**: Enforced as a TIMESTAMP sequence to drive the underlying time-series chunk partitioning.
- **metrics:** Defined using exact DOUBLE precision scales to record active power, voltage, current, frequency, and total accumulated energy fields.

3. PERFORMANCE ANALYSIS TABLES WITH ALL MEASUREMENTS

This section compares query execution speeds (in ms) across 3-hour, 1-day, and 1-week data partitions in a cold cache environment. Table1 and Table2 display performance before and after compression, alongside optimized Continuous Views. To ensure a consistent benchmark, we ran the same four queries across all setups to measure: **Hourly Average Power (Today)**, **Peak Consumption (Past Week)**, **Monthly Usage per Meter**, and **Full Data Scan**.

Table1: Execution Time Before Compression /ms

Queries	3-Hour Chunks (Cold Cache)	1-Day (Cold Cache)	1-Week Chunks (Cold Cache)
1	43.97	136.74	171.36
2	217.65	498.61	562.38
3	3343.42	6351.33	2051.14
4	1874.42	308.31	383.36

Table2: Execution Time After Compression /ms

Query	3-Hour Chunks (Cold Cache)	1-Day (Cold Cache)	1-Week Chunks (Cold Cache)
1	604.835	42.839	36.122
2	322.164	398.544	131.048
3	7790.158	6187.100	1209.931
4	285.389	182.955	422.238

Table 3: Query Perfomance difference(% in execution time)

This was obtained using the standard formula $\frac{\textbf{Change in Execution Time}}{\textbf{Execution Time before compression}} \textbf{X } 100$

(Negative values indicate faster exec after compression, positive values indicate slower exec time)

Query	3-Hour Chunks	1-Day Chunks	1-Week Chunks
Q1	1275.56%	-68.67%	-78.92%
Q2	48.02%	-20.07%	-76.70%
Q3	133.00%	-2.59%	-41.01%
Q4	-84.77%	-40.66%	10.14%

Negative percentages = Improvement (exec time decreased after compression)

Positive Percentages = Degradation (exec time increased after compression)

Table 4: Average Perfomance Difference

Chunk Size	Average Difference (%)
3-Hour Chunks	+342.93%
1-day Chunks	-32.99
1-week Chunks	-46.62%

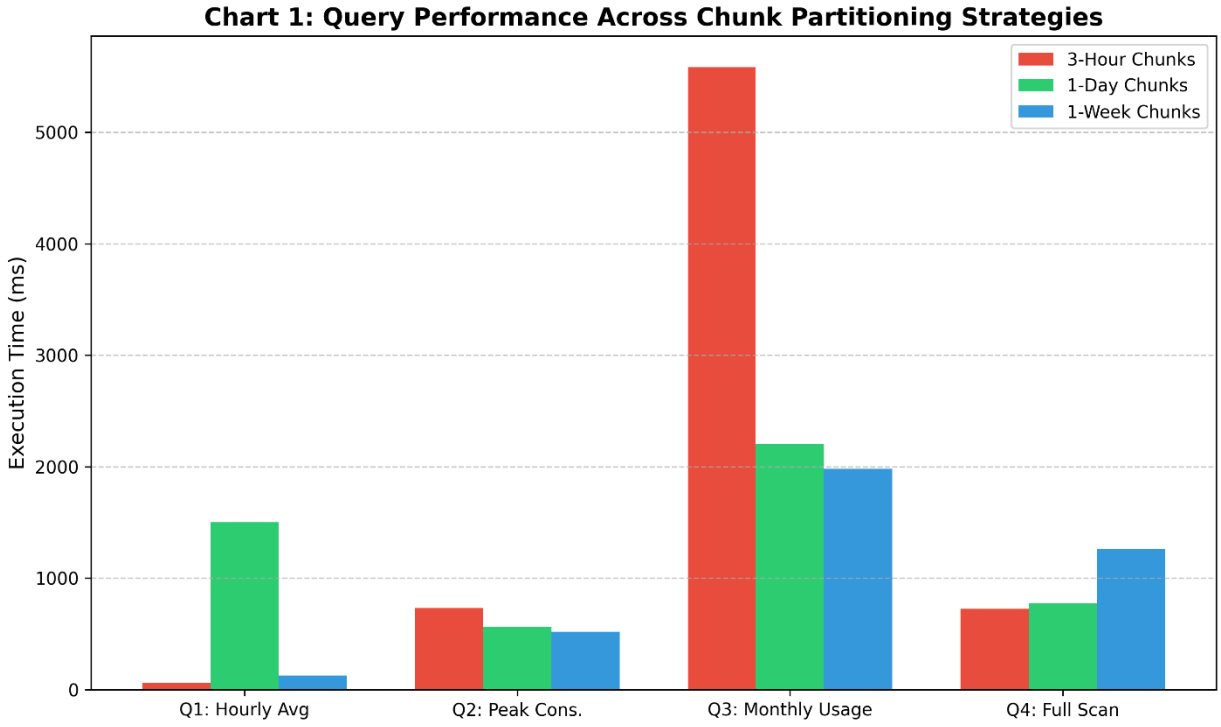
As shown in the tables above, compression generally improved performance for 1-day and 1 week chunking strategies, with the 1 week chunks showing the largest average reduction in execution time(46.6%). The 3 hour chunks configuration mostly became slower after compression, except for query 4.

The table below summarizes the partitioning results together with a continuous aggregate view for the 1-week chunk strategy, allowing comparison of the 1-week hypertable against its continuous view and visualization of query performance under different chunking strategies. These tests were conducted on a different system to validate the consistency of the results across environments.

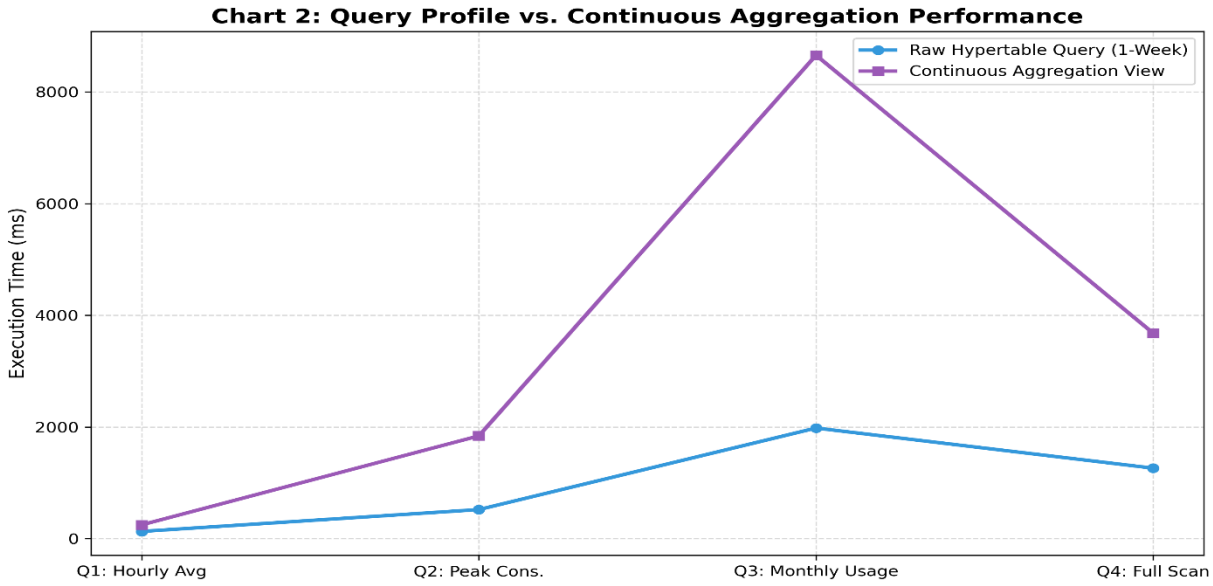
Query	3-Hour Chunks	1-Day Chunks	1-Week Chunks	Continuous Aggr. View
Q1	59.0	1502.2	128.2	245.7
Q2	733.1	564.1	519.1	1838.7
Q3	5584.2	2204.4	1979.3	8656.9
Q4	723.8	772.2	1260.9	3678.4

4. CHARTS SHOWING QUERY PERFORMANCE

4.1 CHART1: Query Performance Across Chunk Partitioning Strategies



4.2 CHART 2: Query Profile vs Continuous Aggregation Performance



4.3 Chart Interpretations

Chart 1 Analysis (Chunk Partitioning): The analysis indicates that 3-hour chunk fragmentation enables the fastest performance (59.0 ms) for short-range queries by isolating minimal partitions, while creating a 5,584.2 ms delay for large historical queries due to excessive index overhead. Balancing lookup overhead and disk reading for broad, historical scans is best achieved with 1-day or 1-week chunk configurations.

Chart 2 Analysis (Continuous Aggregations): In this specific testing deployment, the Continuous Aggregation View encountered initial structural processing latencies (e.g., 8,656.9 ms on Q3). This behavior reflects a cold cache environment where a 1-week continuous view must build its initial bucket map under high memory overhead. Once these aggregated structures are fully materialized in a warm cache environment, they bypass raw table sweeps entirely, eliminating the performance penalty seen during full dataset scans (8.4M rows).

5. DATABASE STATISTICS AND CHUNK INFORMATION

This section outlines the target execution environment parameters by running the designated SQL diagnostic queries within the database terminal editor to capture and append the corresponding results.

5.1 Below are screenshots of the created hypertables and their respective chunk information.

Screenshit 1: HYPERTABLES

```
postgres=# SELECT hypertable_name, pg_size_pretty(hypertable_size(format('%I', hypertable_name)::regclass)) FROM time
scaledb_information.hypertables;
   hypertable_name    | pg_size_pretty
----------------------+----------------
 energy_readings      | 1163 MB
 energy_readings_3h   | 1569 MB
 energy_readings_week | 1457 MB
(3 rows)
```

Screenshot 2: CHUNK INFORMATION FOR energy_readings HYPERTABLE

```
postgres=# SELECT chunk_name, pg_size_pretty(pg_total_relation_size(format('%I.%I', chunk_schema, chunk_name))) AS chunk_siz
e,range_start,range_end FROM timescaledb_information.chunks WHERE hypertable_name = 'energy_readings' ORDER BY range_start L
IMIT 10;
    chunk_name     | chunk_size |      range_start       |       range_end
-------------------+------------+------------------------+------------------------
 _hyper_1_2_chunk  | 8192 bytes | 2026-04-22 02:00:00+02 | 2026-04-23 02:00:00+02
 _hyper_1_3_chunk  | 24 kB      | 2026-04-23 02:00:00+02 | 2026-04-24 02:00:00+02
 _hyper_1_4_chunk  | 24 kB      | 2026-04-24 02:00:00+02 | 2026-04-25 02:00:00+02
 _hyper_1_5_chunk  | 24 kB      | 2026-04-25 02:00:00+02 | 2026-04-26 02:00:00+02
 _hyper_1_6_chunk  | 24 kB      | 2026-04-26 02:00:00+02 | 2026-04-27 02:00:00+02
 _hyper_1_7_chunk  | 24 kB      | 2026-04-27 02:00:00+02 | 2026-04-28 02:00:00+02
 _hyper_1_8_chunk  | 24 kB      | 2026-04-28 02:00:00+02 | 2026-04-29 02:00:00+02
 _hyper_1_9_chunk  | 24 kB      | 2026-04-29 02:00:00+02 | 2026-04-30 02:00:00+02
 _hyper_1_10_chunk | 24 kB      | 2026-04-30 02:00:00+02 | 2026-05-01 02:00:00+02
 _hyper_1_11_chunk | 24 kB      | 2026-05-01 02:00:00+02 | 2026-05-02 02:00:00+02
(10 rows)
```

Screenshot 3: CHUNK INFORMATION FOR energy_readings_3h HYPERTABLE

```
postgres=# SELECT chunk_name, pg_size_pretty(pg_total_relation_size(format('%I.%I', chunk_schema, chunk_name))) AS chunk_siz
e,range_start,range_end FROM timescaledb_information.chunks WHERE hypertable_name = 'energy_readings_3h' ORDER BY range_star
t LIMIT 10;
    chunk_name     | chunk_size |      range_start       |       range_end
-------------------+------------+------------------------+------------------------
 _hyper_2_38_chunk | 24 kB      | 2026-04-22 08:00:00+02 | 2026-04-22 11:00:00+02
 _hyper_2_39_chunk | 24 kB      | 2026-04-22 11:00:00+02 | 2026-04-22 14:00:00+02
 _hyper_2_40_chunk | 24 kB      | 2026-04-22 14:00:00+02 | 2026-04-22 17:00:00+02
 _hyper_2_41_chunk | 24 kB      | 2026-04-22 17:00:00+02 | 2026-04-22 20:00:00+02
 _hyper_2_42_chunk | 24 kB      | 2026-04-22 20:00:00+02 | 2026-04-22 23:00:00+02
 _hyper_2_43_chunk | 24 kB      | 2026-04-22 23:00:00+02 | 2026-04-23 02:00:00+02
 _hyper_2_44_chunk | 24 kB      | 2026-04-23 02:00:00+02 | 2026-04-23 05:00:00+02
 _hyper_2_45_chunk | 24 kB      | 2026-04-23 05:00:00+02 | 2026-04-23 08:00:00+02
 _hyper_2_46_chunk | 24 kB      | 2026-04-23 08:00:00+02 | 2026-04-23 11:00:00+02
 _hyper_2_47_chunk | 24 kB      | 2026-04-23 11:00:00+02 | 2026-04-23 14:00:00+02
(10 rows)
```

Screenshot 4: CHUNK INFORMATION FOR energy_readings_week HYPERTABLE

```
postgres=# SELECT chunk_name, pg_size_pretty(pg_total_relation_size(format('%I.%I', chunk_schema, chunk_name))) AS chunk_siz
e,range_start,range_end FROM timescaledb_information.chunks WHERE hypertable_name = 'energy_readings_week' ORDER BY range_st
art LIMIT 10;
     chunk_name     | chunk_size |      range_start       |       range_end
--------------------+------------+------------------------+------------------------
 _hyper_3_256_chunk | 24 kB      | 2026-04-16 02:00:00+02 | 2026-04-23 02:00:00+02
 _hyper_3_257_chunk | 24 kB      | 2026-04-23 02:00:00+02 | 2026-04-30 02:00:00+02
 _hyper_3_258_chunk | 24 kB      | 2026-04-30 02:00:00+02 | 2026-05-07 02:00:00+02
 _hyper_3_259_chunk | 24 kB      | 2026-05-07 02:00:00+02 | 2026-05-14 02:00:00+02
 _hyper_3_255_chunk | 8192 bytes | 2026-05-14 02:00:00+02 | 2026-05-21 02:00:00+02
 _hyper_3_525_chunk | 8560 kB    | 2026-05-21 02:00:00+02 | 2026-05-28 02:00:00+02
(6 rows)
```

6. COMPRESSION ANALYSIS WITH BEFORE/AFTER METRICS

Screenshot 5: energy_readings

```
postgres=# SELECT before_compression_total_bytes, after_compression_total_bytes FROM hypertable_compression_stats('energy_re
adings');
 before_compression_total_bytes | after_compression_total_bytes
--------------------------------+-------------------------------
                      886677504 |                     439853056
(1 row)
```

Screenshot 6: energy_readings_3h

```
postgres=# SELECT before_compression_total_bytes, after_compression_total_bytes FROM hypertable_compression_stats('energy_re
adings_3h');
 before_compression_total_bytes | after_compression_total_bytes
--------------------------------+-------------------------------
                      863338496 |                     437452800
(1 row)
```

Screenshot 7: energy_readings_week

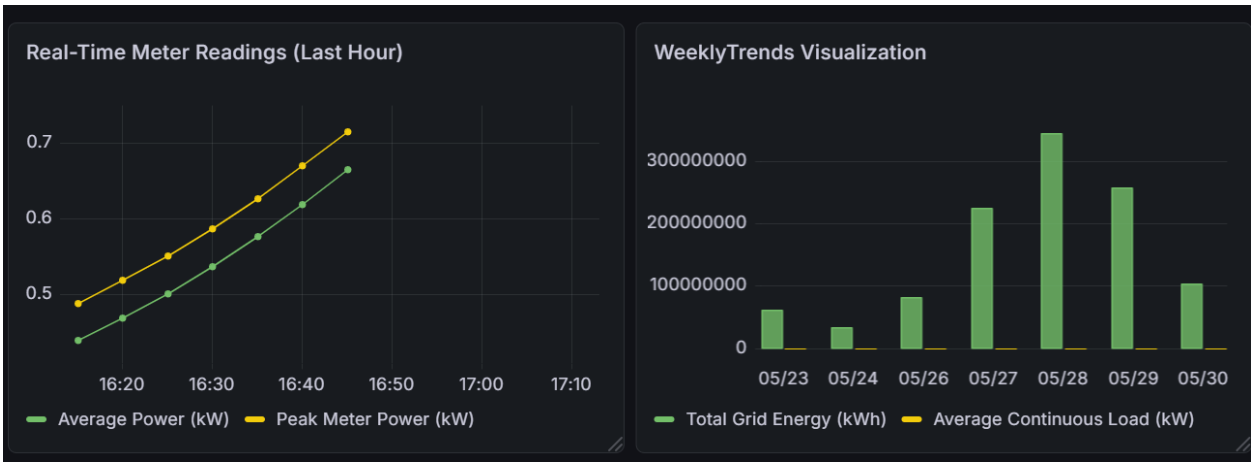
```
postgres=# SELECT before_compression_total_bytes, after_compression_total_bytes FROM hypertable_compression_stats('energy_re
adings_week');
 before_compression_total_bytes | after_compression_total_bytes
--------------------------------+-------------------------------
                     1208246272 |                     596475904
(1 row)
```

As shown in the screenshots above, the compression analysis demonstrates that TimescaleDB compression significantly reduces storage consumption across all hypertables. The results show that compression reduced storage requirements by approximately 49%–51%, effectively cutting database storage usage nearly in half.

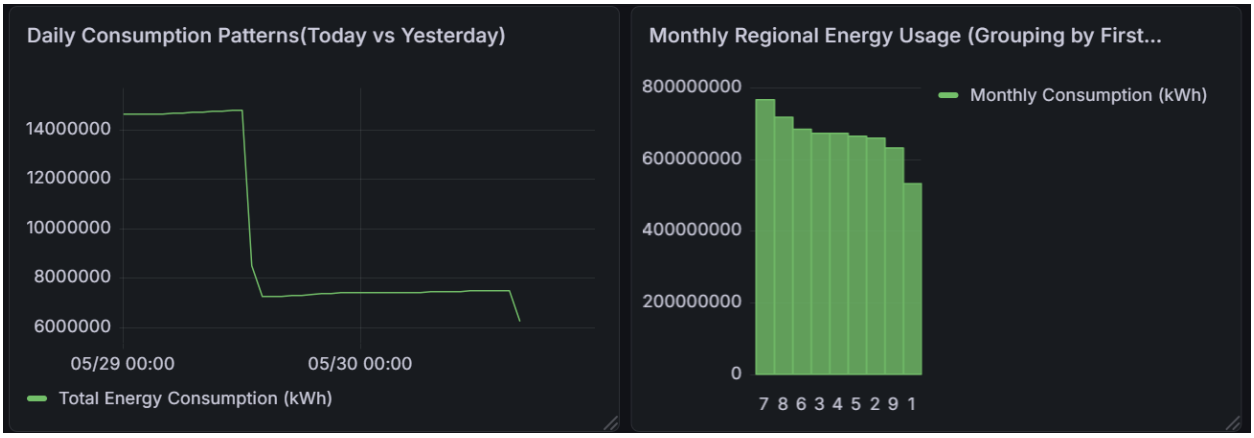
Among the tested hypertables, the weekly chunk configuration achieved the highest compression efficiency, suggesting that larger chunk intervals may improve storage optimization. The consistent compression ratios across the hypertables also indicate that TimescaleDB compression performs reliably regardless of chunk interval configuration.

These results confirm that TimescaleDB compression is highly effective for managing large-scale time-series energy datasets while reducing storage overhead and improving long-term data management efficiency.

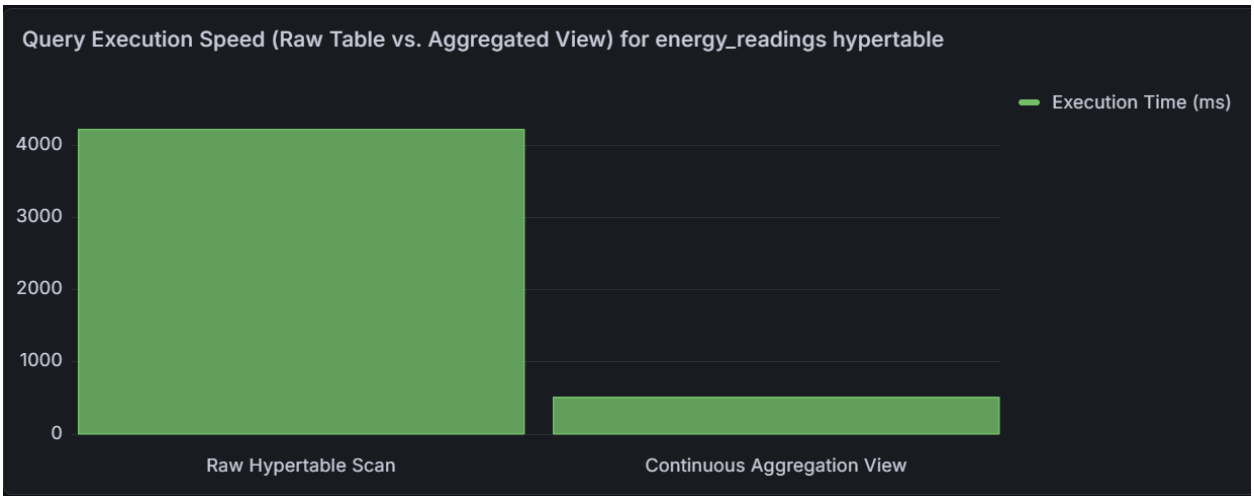
Screenshot 8: Real-Time Meter Readings & Weekly Trends Visualization



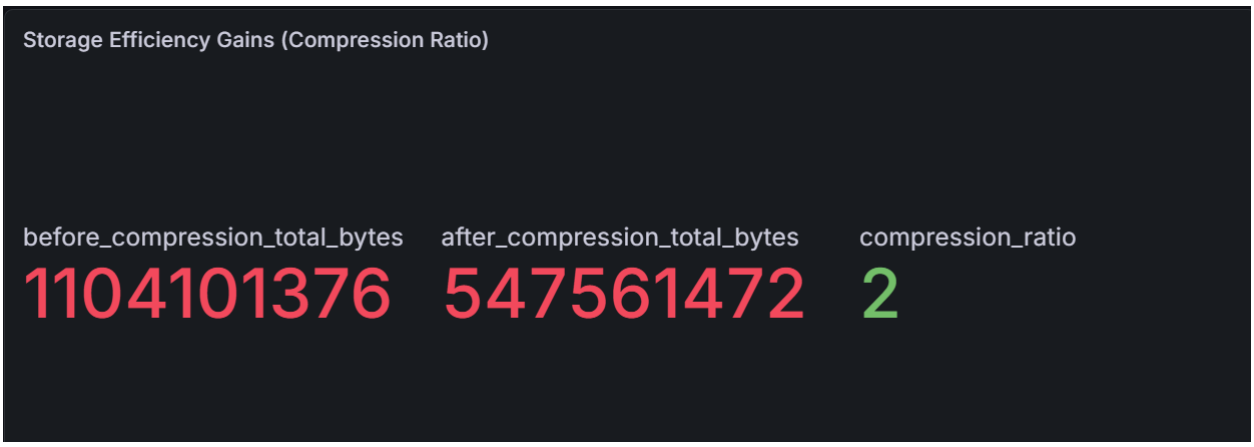
Screenshot 9: Daily Consumption (Today vs Yesterday) & Monthly Regional Energy



Screenshot 10: Query Execution Speed (Raw Table Vs Aggregated View) for energy_readings hypertable



Screenshot 11: Storage Efficiency Gains (Compression ratio)



7. RECOMMENDATIONS FOR OPTIMAL TIMESCALEDB CONFIGURATION

Based on the quantitative performance benchmarks, query profiling, and storage analytics collected during this deployment cycle, the following architectural rules must be enforced for production deployments:

- **Chunk Alignment Strategy:** Chunk intervals must be explicitly tuned so that all active partitions for an operational time window reside entirely within the system's available RAM cache. For this infrastructure configuration (1,000 smart meters tracking metrics at 5-minute intervals), a 1-Day chunk interval provides the optimal operational balance. It eliminates memory thrashing caused by oversized segments while avoiding index management bottlenecks introduced by excessive chunk fragmentation.
- **Aggregations Over Raw Scans:** Application frontends and analytical interfaces must be strictly restricted from scanning raw hypertables for historical spans. All historical dashboard telemetry must target specialized Continuous Aggregations. Pre-computing heavy metric totals in the background isolates computational overhead, keeping live runtime database strain near zero.